

SUMMER 2026



LONGHORN HAPKIT

Lab guide #1

ROSS NEUMAN

Assistant Professor of Practice, The University of Texas at Austin

Get a box

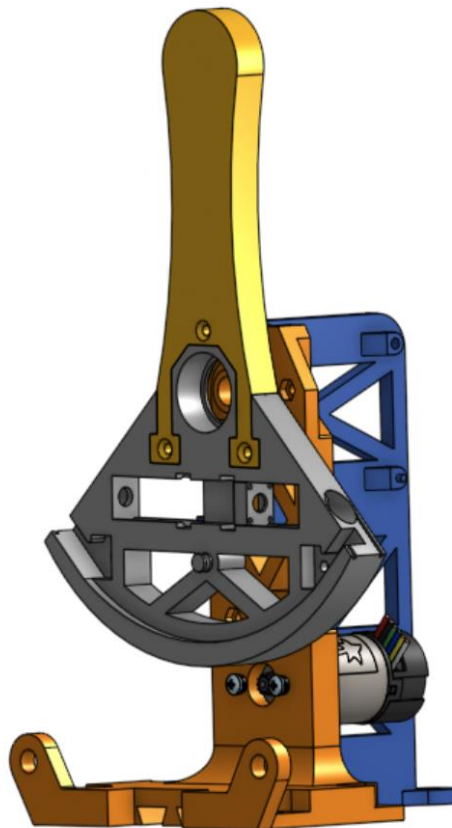
- Feel free to trade around for different colors
 - Colors are all random
 - All prints with same material
- Some prints may have support material hanging off, feel free to remove
- If you suspect you have a defective part let me know
- Label your box (sharpies will be passed around)

Practices for Part 1 of Camp

- I am only including screenshots of code
 - I want you to have practice entering lines of code, this will help you retain what you use
 - I will upload the code we discuss today after we are through so you can reference it as the week progresses
- Please don't use LLMs (ChatGPT, Claude, etc.)
 - I want you to use your own brains as we get our robots put together
 - Later in the week when things are more open-ended, you are welcome to use these tools to bring your ideas to life
- Please *do* use your neighbors
 - Make friends, help each other out, exchange ideas!

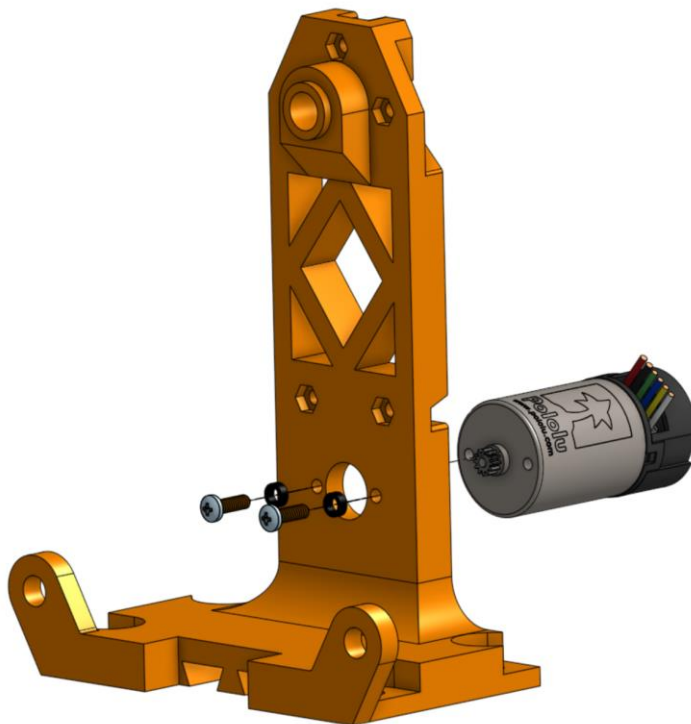
What's in the Box?

- 3D printed parts
 - Base
 - PCB Mount
 - Sector Pulley
 - Handle
 - Bearing Housing
 - Motor Shaft
- Arduino Uno
- Polulu 12V Motor + Encoder
- L298N Motor Driver
- 12V DC power supply
- Tensioning slider+cable
- 2 Ball bearings
- Bag of hardware
- Screwdriver (Philips #1)
- Tiny Screwdriver
- 4mm Hex Driver (aka Allen wrench)
- Wires
- Bag of Hardware



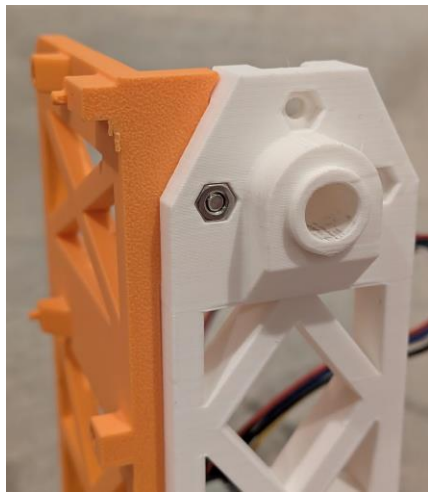
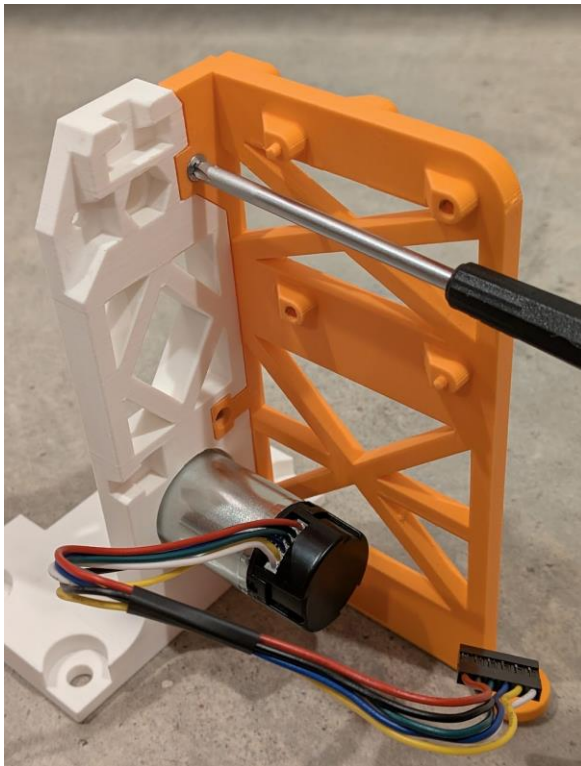
Assemble! Mount Motor to Base

- Use the 2 round-head screws along with the 2 spacers and secure the motor to the base
 - It will help later if you do this with the motor wires oriented upward
- You may need to drive the screws to get them through the mounting holes



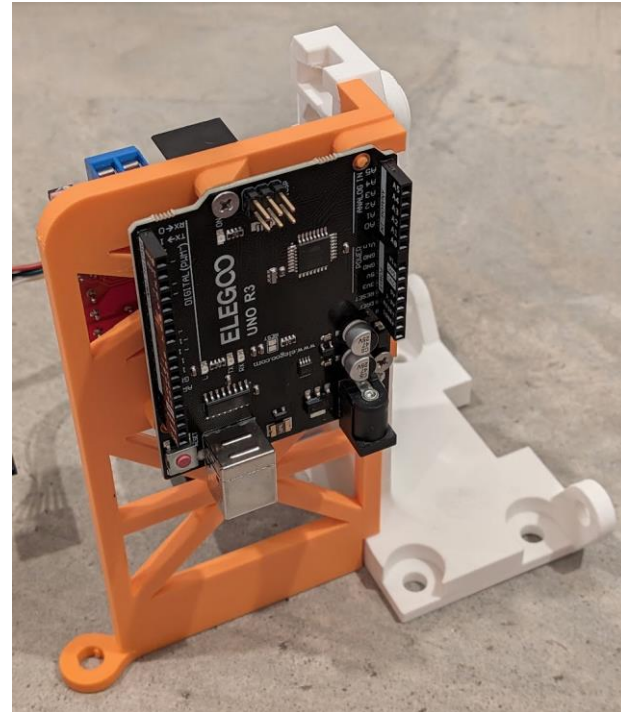
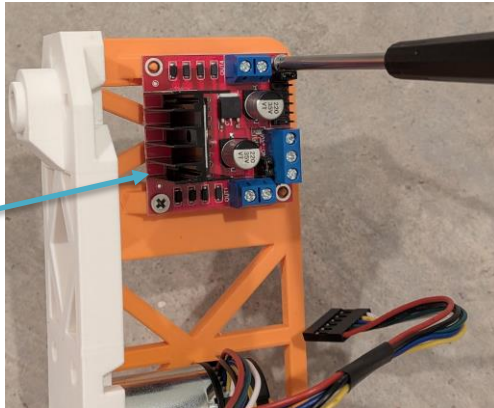
Connect the Base and PCB Mount

- Slot the PCB mount into the Base
- Secure with one M3 screw and one M3 Nut
 - You can make it more secure tomorrow when I have more screws (whoops)
- Should sit flat on your desk now



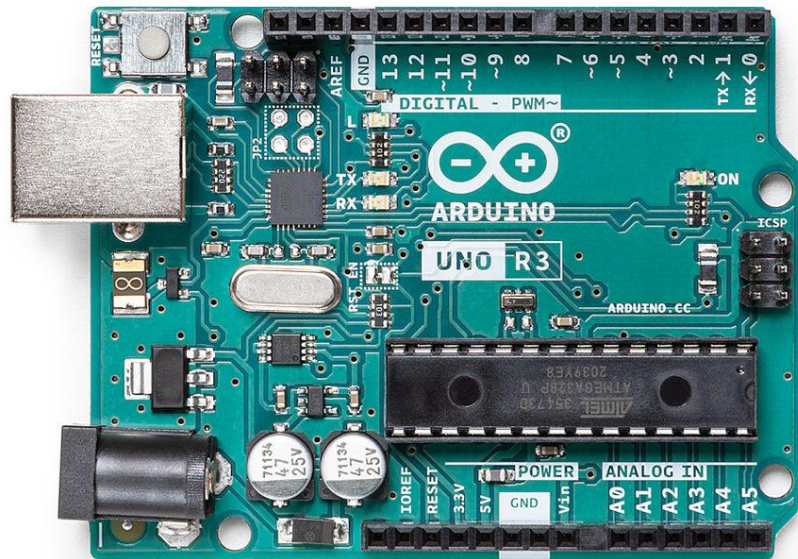
Attach Arduino and Driver Board

- Arduino and driver board screw into PCB mount using 2 M3 screws each
- Orient the driver board such that the heat sink is facing inward
- DO NOT OVERTIGHTEN
 - You are *tapping* your own threads into the plastic,
 - If you try to tighten too much, you will strip these threads and lose all holding power



What is Arduino?

- Arduino is a **microcontroller development board** that allows easy interfacing with the chip on board
- In industrial applications, PCBs would be designed specifically for the given **embedded system**, but this allows for reconfiguring/tinkering/exploring
- Our Arduino Uno is based around the ATmega328P, an 8-bit microprocessor

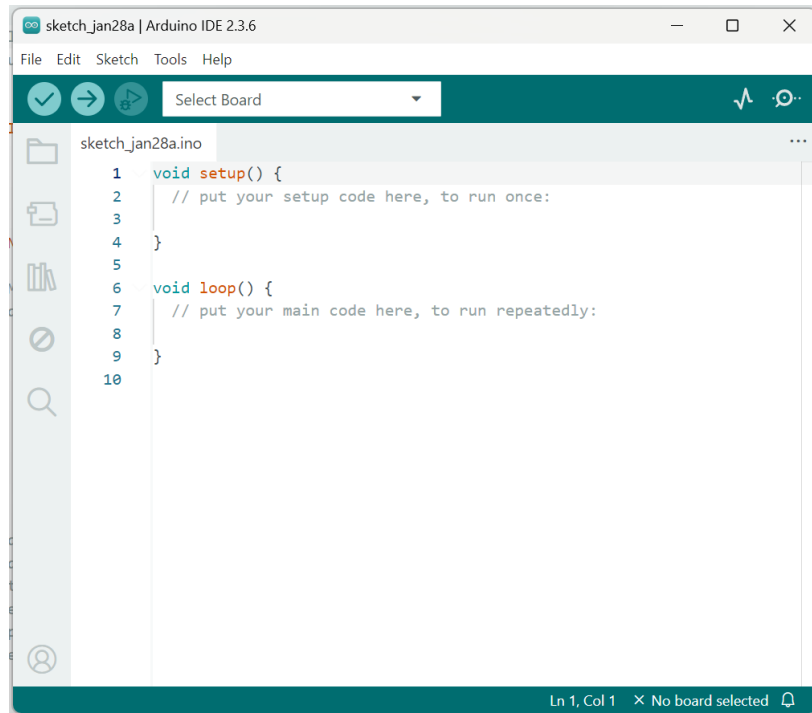


Arduino IDE

- Open up a terminal on your computer and type “arduino” then press enter
- You should see the Arduino IDE open up
 - This is where you will write programs to upload to your Arduino
- Connect your Arduino to your computer with the included USB cable
- Select ttyUSB, search for “UNO” and select Arduino UNO as your board for this port

Programming Arduino

- Arduino has an IDE that you write programs in
- Beginner-friendly wrapper on C++
 - You can program straight up C++ on the Arduino if you like
- You write a single program (“sketch”) that will run continuously on the board whenever it is powered
 - `setup()` runs once at the beginning
 - `loop()` the runs continuously as long as the device remains powered on
 - This is known as a “superloop”



Blink—The “Hello World” of Arduino

- Arduino has an onboard LED that you can program
- It is a pin named with the constant *LED_BUILTIN*
- In your **setup** section, initialize it as an output pin:

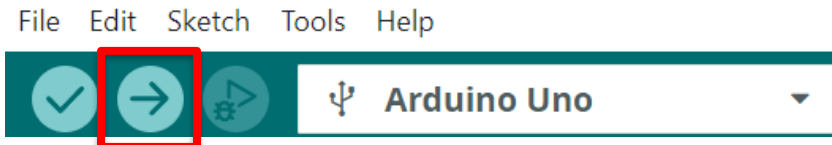
```
1 void setup() {  
2     // put your setup code here, to run once:  
3     pinMode(LED_BUILTIN, OUTPUT); //initialize the LED pin as an output  
4 }
```

Blink—The “Hello World” of Arduino

- Now write your loop
- `digitalWrite()` lets you set your pin to HIGH or to LOW
 - This is your 1 or 0
 - Electrically, it will set the pin to either +5V or 0V
- To turn LED on, send voltage HIGH command
- To turn LED off, send voltage LOW command
- To wait for a given number of milliseconds, use the “delay” function

```
6  void loop() {
7    // put your main code here, to run repeatedly:
8    digitalWrite(LED_BUILTIN, HIGH); //turn LED on by sending voltage HIGH command
9    delay(1000);                      //wait for 1000ms (one second)
10   digitalWrite(LED_BUILTIN, LOW);   //turn LED off by sending voltage LOW command
11   delay(1000);                      //wait for 1000ms
```

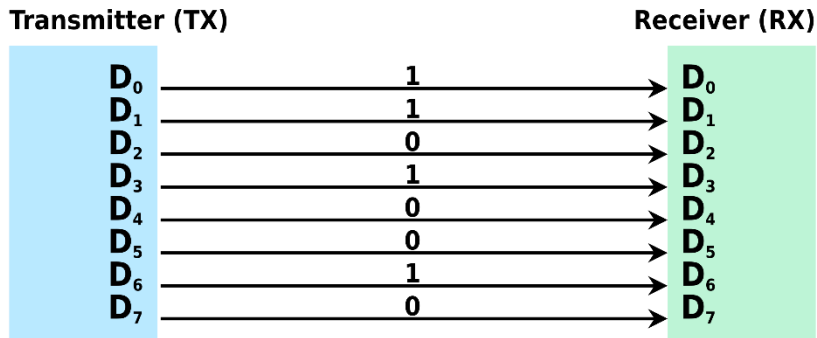
- Upload it and watch your LED blink!



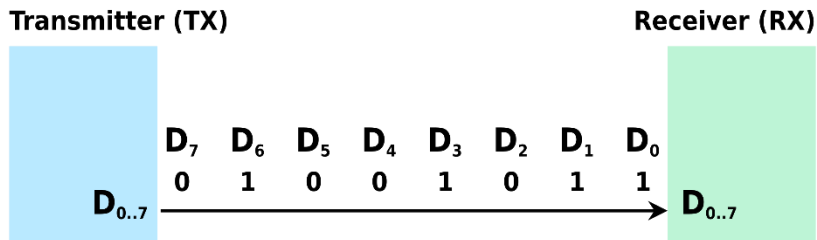
Talking to your Arduino

- Arduino uses **serial communication** to share information with other devices
- Serial sends less data per clock cycle than parallel, but simplicity of transmission allows much higher clock rates
- “Serial monitor” in Arduino IDE allows you to read information from the board in real time
 - Baud rate up to 115200 in serial monitor

Parallel interface example



Serial interface example



Speed of Serial -- Baud

- “Symbols per second” or pulses per second of a transmission medium
- Émile Baudot invented “Baudot code” for early telegraphy
- Operator would enter one symbol for each “click” of the distributor in a cadence
- The “Baud” was the measure of how quickly these symbols would be entered in
- Name stuck around for ensuing serial communications

(No Model.)

No. 388,244.

J. M. E. BAUDOT.
PRINTING TELEGRAPH.

11 Sheets—Sheet 6.

Patented Aug. 21, 1888.

Fig. 24.



	1	2	3	4
A	+	-	-	+
B	+	-	+	+
C	+	+	+	+
D	+	+	+	+
E	+	+	+	+
F	+	+	+	+
G	+	+	+	+
H	+	+	+	+
I	+	+	+	+
J	+	+	+	+
K	+	+	+	+
L	+	+	+	+
M	+	+	+	+
N	+	+	+	+
O	+	+	+	+
P	+	+	+	+
Q	+	+	+	+
R	+	+	+	+
S	+	+	+	+
T	+	+	+	+
U	+	+	+	+
V	+	+	+	+
W	+	+	+	+
X	+	+	+	+
Y	+	+	+	+
Z	+	+	+	+
0	+	+	+	+
1	+	+	+	+
2	+	+	+	+
3	+	+	+	+
4	+	+	+	+
5	+	+	+	+
6	+	+	+	+
7	+	+	+	+
8	+	+	+	+
9	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+	+	+	+
!	+	+	+	+
@	+	+	+	+
#	+	+	+	+
\$	+	+	+	+
%	+	+	+	+
&	+	+	+	+
'	+	+	+	+
(	+	+	+	+
)	+	+	+	+
*	+	+	+	+
+	+	+	+	+
-	+	+	+	+
.	+			

Adding Serial Communication

- If you want to send messages to the serial monitor in your Arduino IDE, type `Serial.begin(desired baud)` in the setup portion of your program

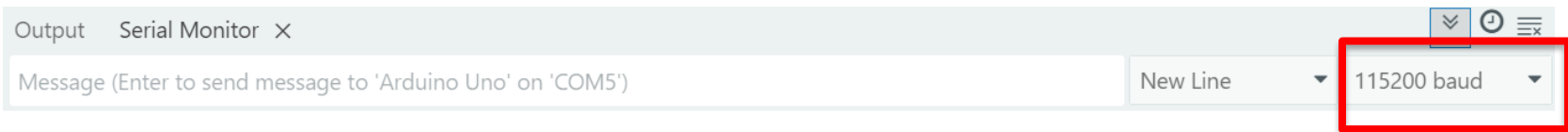
```
void setup() {  
    // put your setup code here, to run once:  
    pinMode(LED_BUILTIN, OUTPUT); //initialize the LED pin as an output  
    Serial.begin(115200); //set up serial communication  
}
```

- To output a message, use `Serial.print("your message")`
- To make each message come up on a new line use `Serial.println()`

```
void loop() {  
    // put your main code here, to run repeatedly:  
    digitalWrite(LED_BUILTIN, HIGH);  
    Serial.println("LED is on");  
    delay(1000);  
    digitalWrite(LED_BUILTIN, LOW);  
    Serial.println("LED is off");  
    delay(1000);  
}
```


Serial Communication

- To observe the output, open the Serial Monitor with ctrl+shift+M or under “Tools”
- Make sure to set the BAUD in the monitor to the same thing you declared in your code
 - Might as well set to the fastest at 115200



Declare a Variable

- Declare an integer variable at the very top of your code

```
int numBlinks = 0; //declare your variables up top

void setup() {
  // put your setup code here, to run once:
  pinMode(LED_BUILTIN, OUTPUT); //initialize the LED pin as an output
  Serial.begin(115200); //set up serial communication
}
```

- Increment the variable each loop and report it to serial monitor

```
numBlinks = numBlinks+1;           //count how many blinks have occurred
Serial.print("There have been ");
Serial.print(numBlinks);           //when printing a variable, do not use quotes
Serial.println(" blinks");
}
```

Write a “for” loop

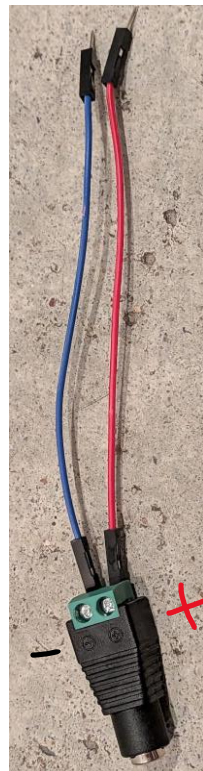
- Use a loop to blink the light only five times

1. **Start value:** `int i = 1`
2. **Condition:** `i <= 5`
3. **Update:** `i++`

```
1 //Now use a loop to blink the light only five times
2 //Note that loop() is empty here, so the blinking happens only in setup
3
4 void setup() {
5     pinMode(LED_BUILTIN, OUTPUT);
6
7     for (int i = 1; i <= 5; i++) {
8         digitalWrite(LED_BUILTIN, HIGH);
9         delay(500);
10
11         digitalWrite(LED_BUILTIN, LOW);
12         delay(500);
13     }
14 }
15
16 void loop() {
17 }
```

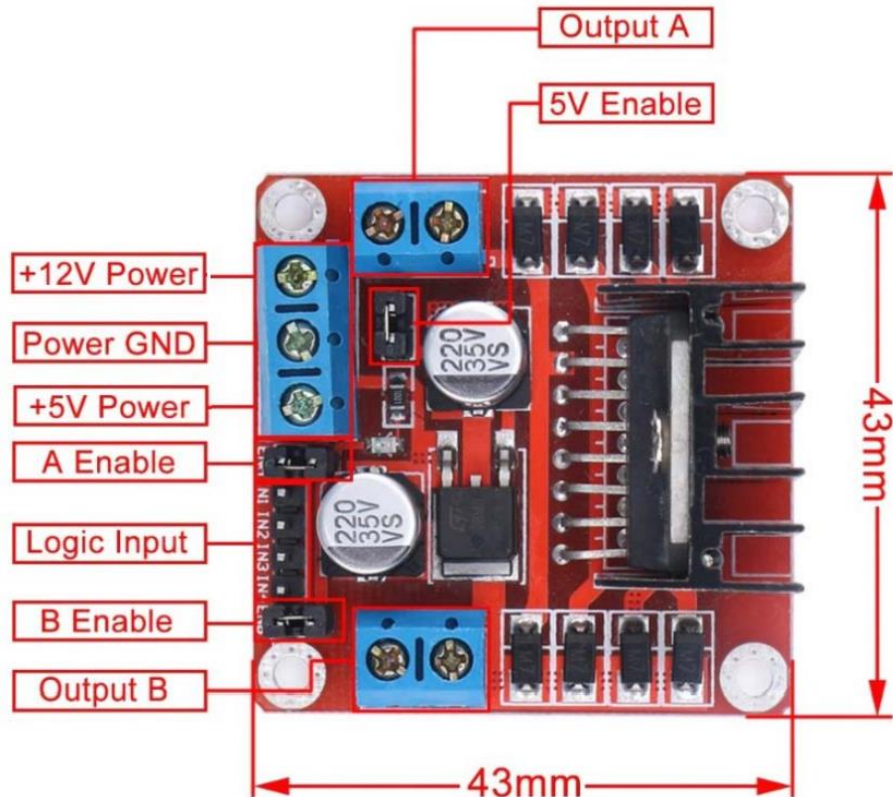
Wiring the Motor - Power

- Attach 10cm M-M wires to the positive and negative terminals of the barrel jack
- Use your tiny screwdriver to secure in place if your regular one doesn't reach the screws
- **Keep track which wire is positive and which is negative**



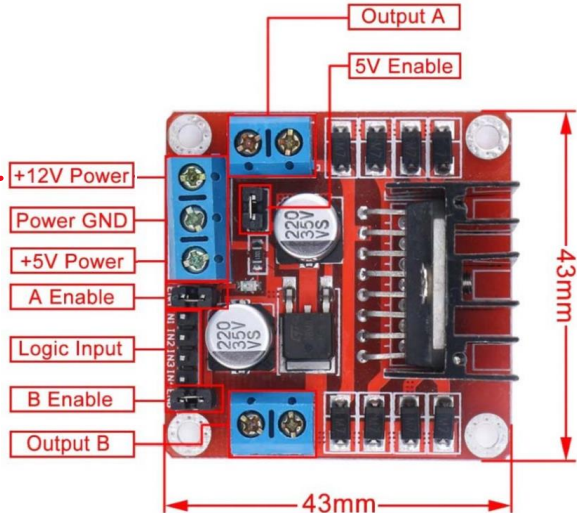
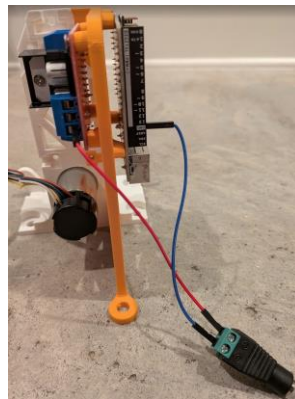
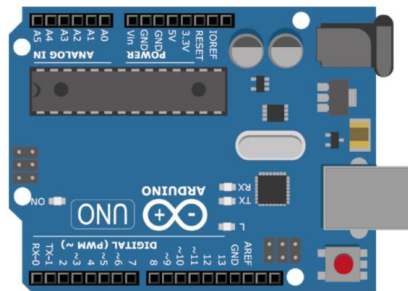
Motor Driver: L298N

- “Drives” motor(s) by providing adequate power and switching signals/directions as needed
 - Arduino has insufficient power to drive the motor
- Our 12V power adaptor will connect to this board



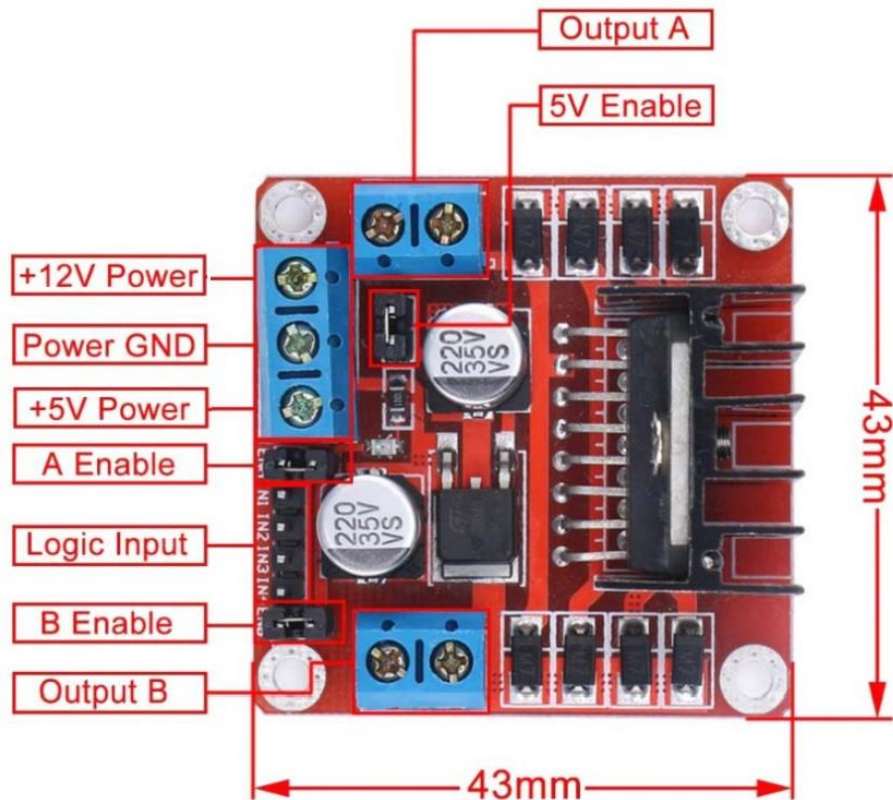
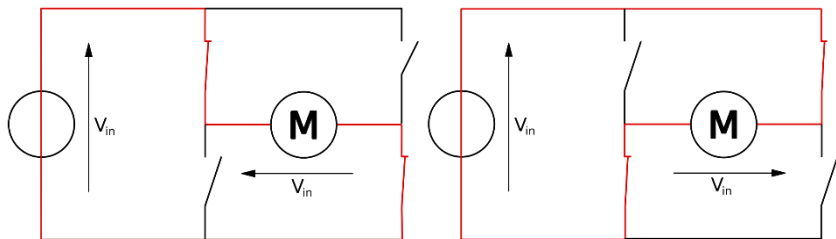
Wiring the Motor - Power

- Attach the positive wire to +12V power on the driver board
- Attach negative to GND on the Arduino
 - You will see in a minute why we're not just running it to Power GND
- **PLEASE TRY NOT TO FLIP THESE** (you could fry your board)



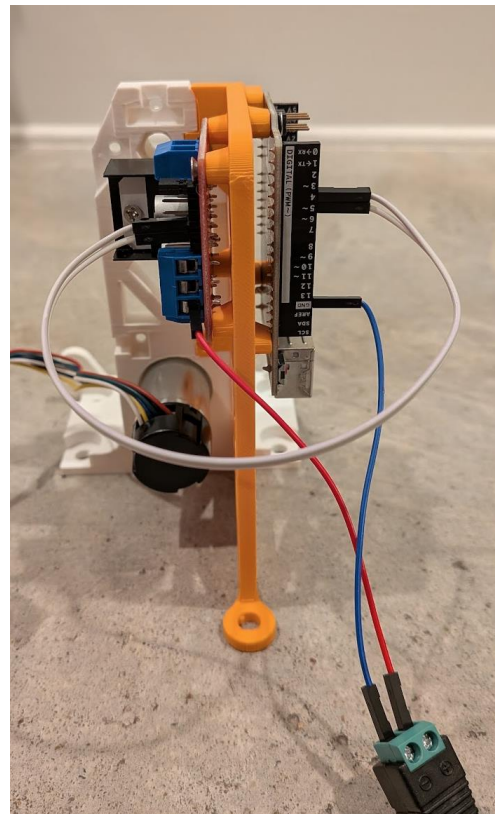
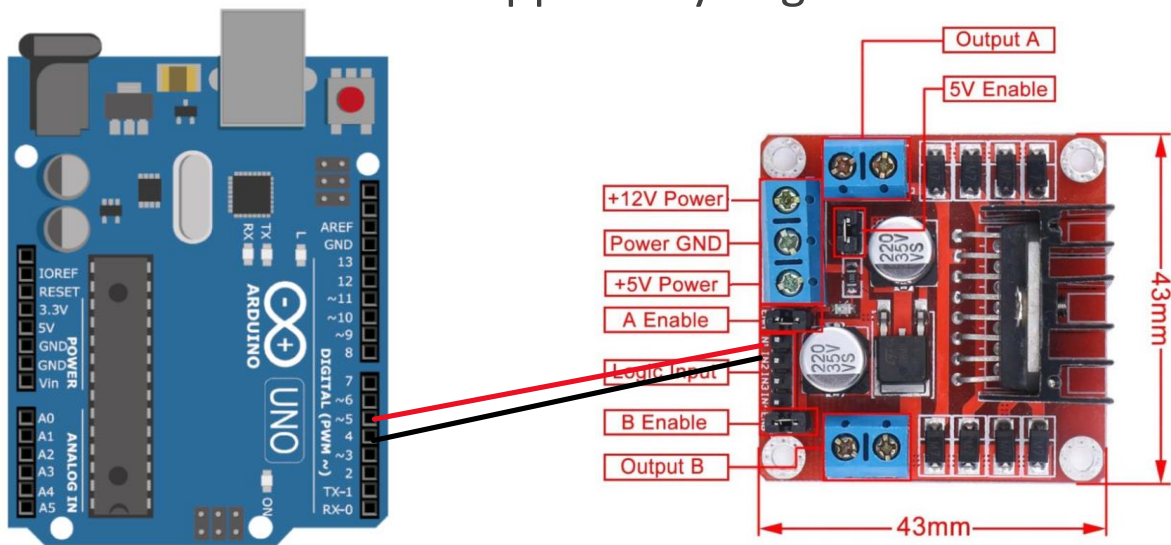
Motor Driver: Logic

- Pins IN1 and IN2 are the logic control for the **H-bridge** circuit
- If IN1 is high and IN2 is low, motor turns one way
- If IN1 is low and IN2 is high, motor turns the other way



Wiring the Motor – Logic Inputs

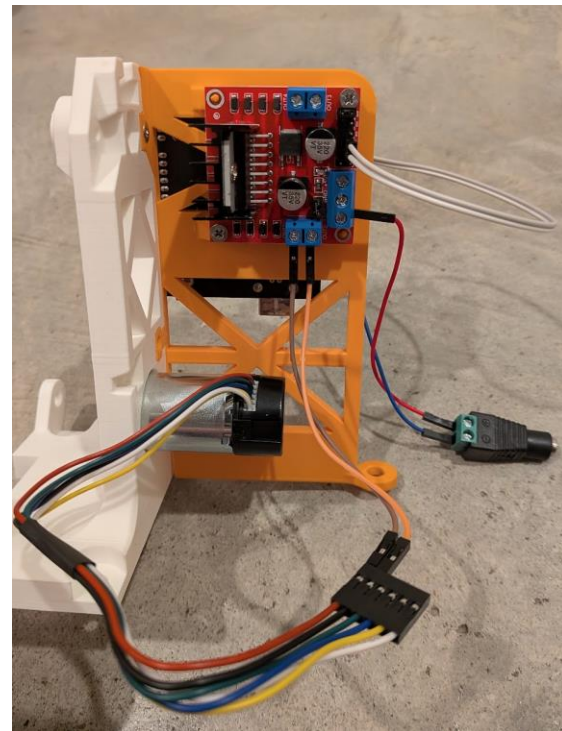
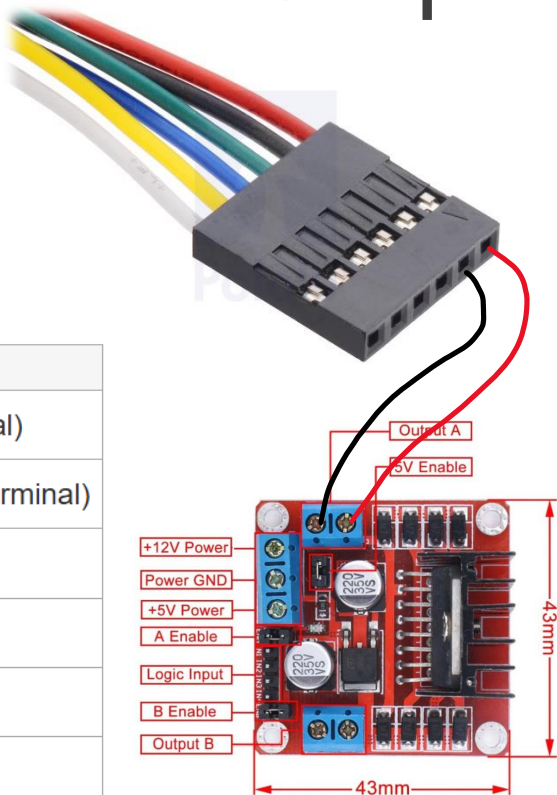
- Use the 20cm F-M wires to:
 - Wire Arduino Pin 5 to IN1
 - Wire Arduino Pin 4 to IN2
- Question: What happens if you get this backwards?



Wiring the Motor - Outputs

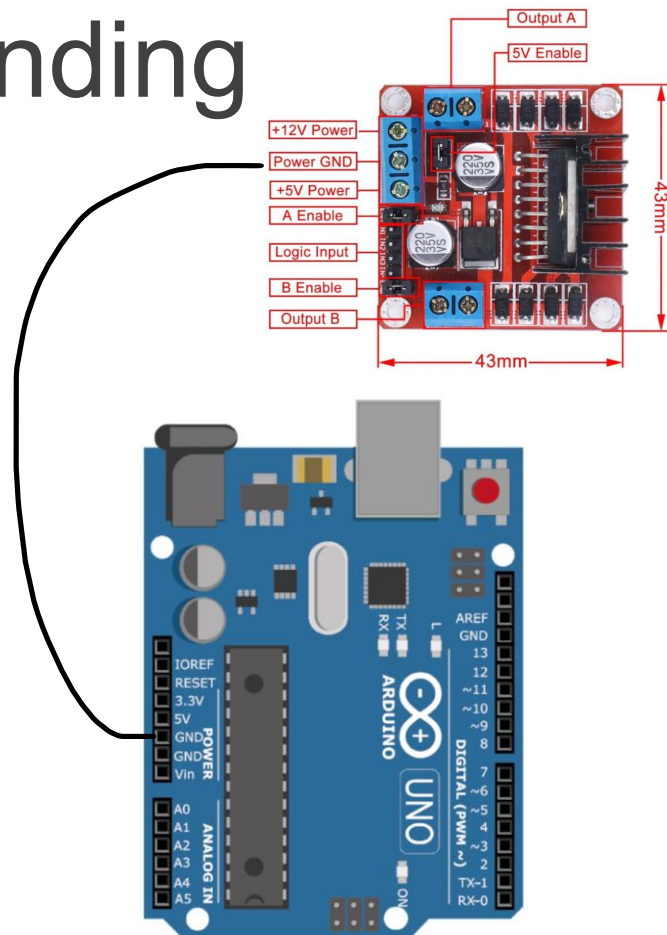
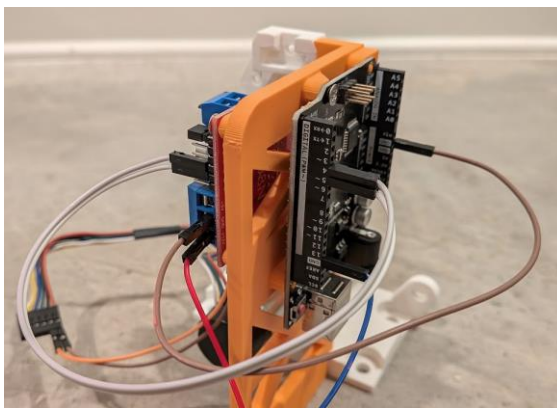
- Use 10cm M-M wires to connect:
 - Red motor power to OUT1
 - Black motor power to OUT2

Color	Function
Red	motor power (connects to one motor terminal)
Black	motor power (connects to the other motor terminal)
Green	encoder GND
Blue	encoder Vcc (3.5 – 20 V)
Yellow	encoder A output
White	encoder B output



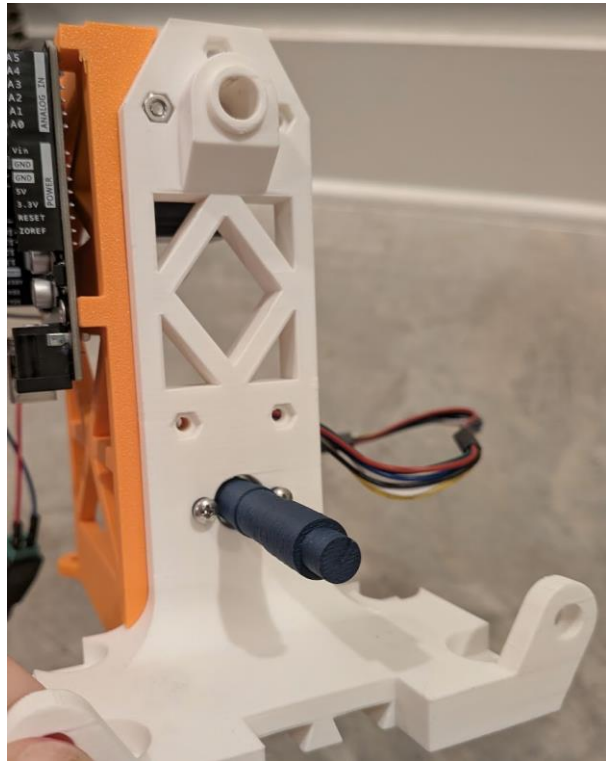
Wiring the Motor - Grounding

- We want the Arduino and the L298N to have a common ground
 - In electronics, we want our circuits to agree on what “0” is, so we tie grounds together
- Use the last 20cm M-M to connect an Arduino GND to the Power GND on the L298N



Attach Motor Shaft

- This might help you visualize what the motor is doing
- Try and get the motor output gear to mesh with the gear profile in the shaft
 - You might have to mash it a bit
 - I have extras if yours gets messed up



Test the Motor

- At this point, we can run the motor forward or backward – so let's try it out!

```
const int MotIn1 = 5; //motor logic pin 1
const int MotIn2 = 4; //motor logic pin 2

void setup() {
  // put your setup code here, to run once:
  pinMode(MotIn1, OUTPUT); //all caps because it's a predefined constant/macro
  pinMode(MotIn2, OUTPUT);

void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(MotIn1, HIGH);
  digitalWrite(MotIn2, LOW);

}
```

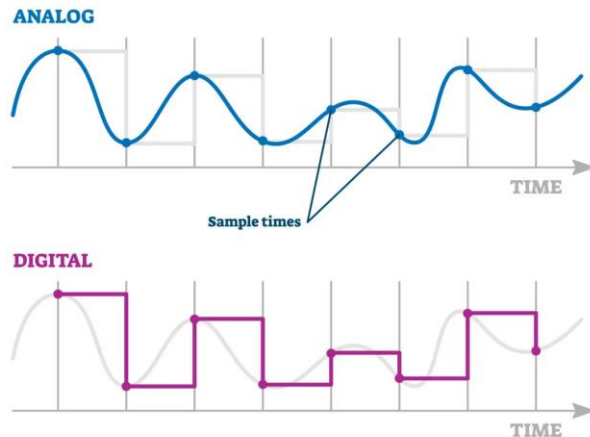
How do we change the speed?

- **Speed** is roughly proportional to the **voltage** across the motor's coils
- Before, we were supplying 12V or 0V to the motor
- How do we get something in between, when our Arduino output pins just write “on” and “off”?

Analog vs. Digital Signals

- Analog signals are **smooth** often arising from physical phenomena
- Digital signals are discrete
 - The more discrete samples in a given window of time, the closer to “analog” you can approximate
- Analog-to-digital conversion is **ADC**
- Digital-to-analog conversion **DAC**
- What might these be useful for?
 - People spend a LOT of money on good DACs and ADCs
- Do you need a DAC to control a motor?

ANALOG VS DIGITAL SIGNAL



The Solution: PWM

- Pulse Width Modulation (PWM) is a way to use digital (on/off) signals to provide “analog” voltages to the motor circuit
- By controlling the proportion of time that the signal is on vs. off, it begins to look like an analog voltage if you switch fast enough



assumes that the average signal is a constant signal

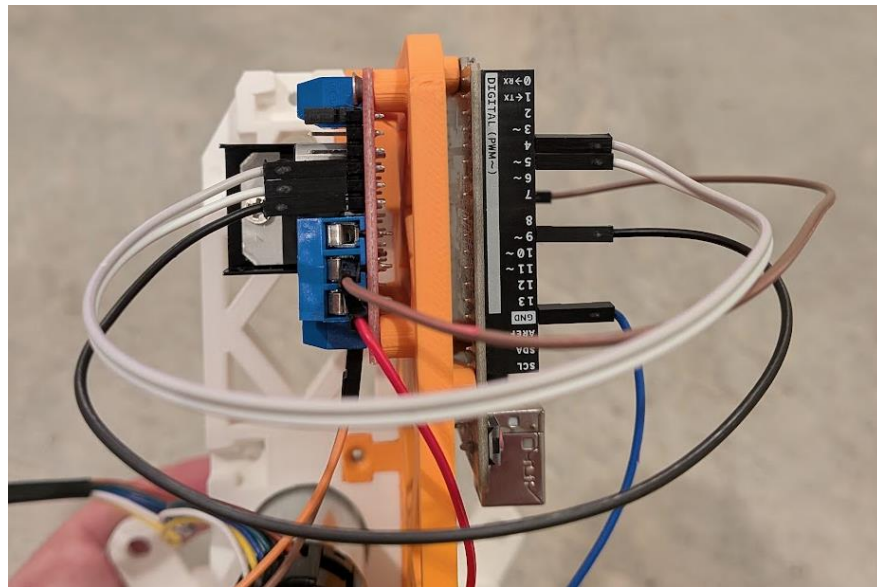


duty cycle is the proportion of **on** time to the **period**



Changing Speeds - PWM

- Remove the jumper from ENA
 - Now we will turn on/off using PWM from arduino
- Use the last F-M 20cm wire to connect ENA to Arduino pin 9
 - Any pin with “~” on Arduino can have PWM output
 - Reason for using pin 9 will be discussed later (for now it doesn't really matter which one you choose)



Test the PWM

- To use PWM in Arduino, use `analogWrite()`
- This takes values between 0 and 255 to dictate the duty cycle

```
1  //use PWM to control the motor speed
2
3  const int MotIn1 = 5; //motor logic pin 1
4  const int MotIn2 = 4; //motor logic pin 2
5  const int pwm = 9;    //motor PWM pin
6
7  void setup() {
8      // put your setup code here, to run once:
9      pinMode(MotIn1, OUTPUT); //all caps because it's a predefined constant/macro
10     pinMode(MotIn2, OUTPUT);
11     pinMode(pwm, OUTPUT);
12
13     digitalWrite(MotIn1, HIGH);
14     digitalWrite(MotIn2, LOW);
15 }
16
17 void loop() {
18     // put your main code here, to run repeatedly:
19     int speed = 50; //this is a value between 0 and 255 to dictate the motor's speed (voltage to the motor)
20     analogWrite(pwm, speed);
21
22 }
```

Write a loop to go through speeds

- Ramp from 0% to 100% duty cycle and back down using for loops

```
1 //use PWM to control the motor speed
2
3 const int MotIn1 = 5; //motor logic pin 1
4 const int MotIn2 = 4; //motor logic pin 2
5 const int pwm = 9;    //motor PWM pin
6
7 void setup() {
8     // put your setup code here, to run once:
9     pinMode(MotIn1, OUTPUT); //all caps because it's a predefined constant/macro
10    pinMode(MotIn2, OUTPUT);
11    pinMode(pwm, OUTPUT);
12
13    digitalWrite(MotIn1, HIGH);
14    digitalWrite(MotIn2, LOW);
15 }
16
17 void loop() {
18
19     for (int speed = 0; speed <= 255; speed = speed + 1) {
20         analogWrite(ENA, speed);
21         delay(20);
22     }
23
24     for (int speed = 255; speed >= 0; speed = speed - 1) {
25         analogWrite(ENA, speed);
26         delay(20);
27     }
28 }
```

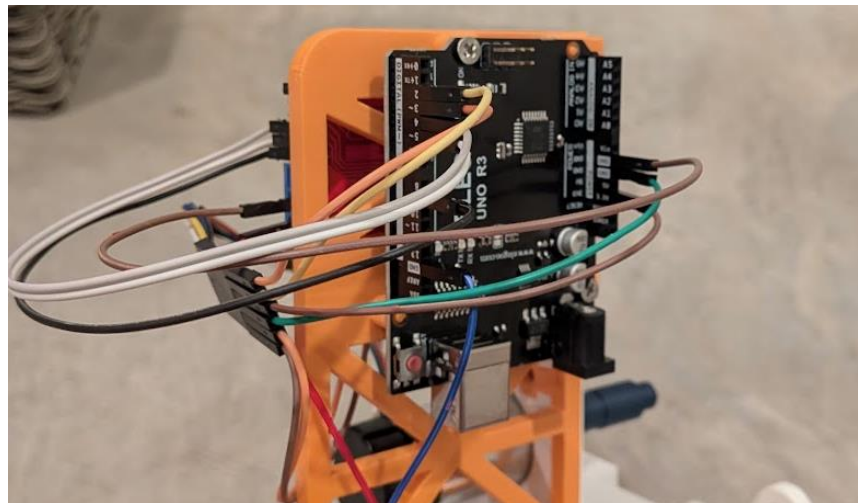
Write a function to handle motor commands

- We want speeds to be positive or negative so the motor can go forward and backward
- Let's write a function to handle this
- Now you can say, e.g. `setMotor(-127)` and the function will tell the motor to spin backward at a 50% duty cycle (roughly 6V to the motor)
- Try to modify the ramp function from the previous slide to ramp from negative to positive and back using `setMotor()` function

```
void setMotor(int motCommand) {  
    // limit values to between -255 and 255  
    if (motCommand > 255) {  
        motCommand = 255;  
    }  
    if (motCommand < -255) {  
        motCommand = -255;  
    }  
  
    //analogWrite commanded speed in the correct direction  
    if (motCommand > 0) {  
        digitalWrite(MotIn1, HIGH);  
        digitalWrite(MotIn2, LOW);  
        analogWrite(pwm, motCommand);  
    }  
    else if (motCommand < 0) {  
        digitalWrite(MotIn1, LOW);  
        digitalWrite(MotIn2, HIGH);  
        analogWrite(pwm, -motCommand);  
    }  
    else {  
        analogWrite(pwm, 0);  
    }  
}
```

Wiring the Encoder

- Connect encoder ground to a ground pin on Arduino
- Connect encoder Vcc to 3.3V pin on the Arduino (5V works fine too)
 - This powers the encoder from the Arduino
- Connect encoder A and encoder B outputs to pins 2 and 3 on Arduino, respectively
 - It doesn't really matter which digital pins you assign these to, but using this scheme will keep us consistent when sharing code

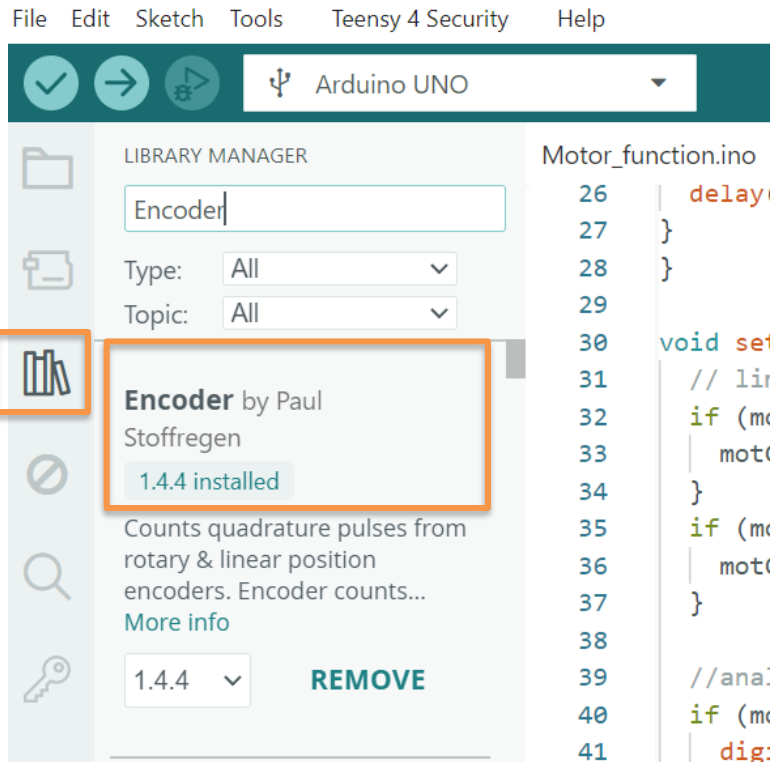


Color	Function
Red	motor power (connects to one motor terminal)
Black	motor power (connects to the other motor termin
Green	encoder GND
Blue	encoder Vcc (3.5 – 20 V)
Yellow	encoder A output
White	encoder B output



Using the Encoder

- We have a library to handle the quadrature encoder math for us:
 - In Arduino IDE open libraries manager (books icon)
 - Search “encoder” and install the Encoder library by Paul Stoffregen



Test it Out

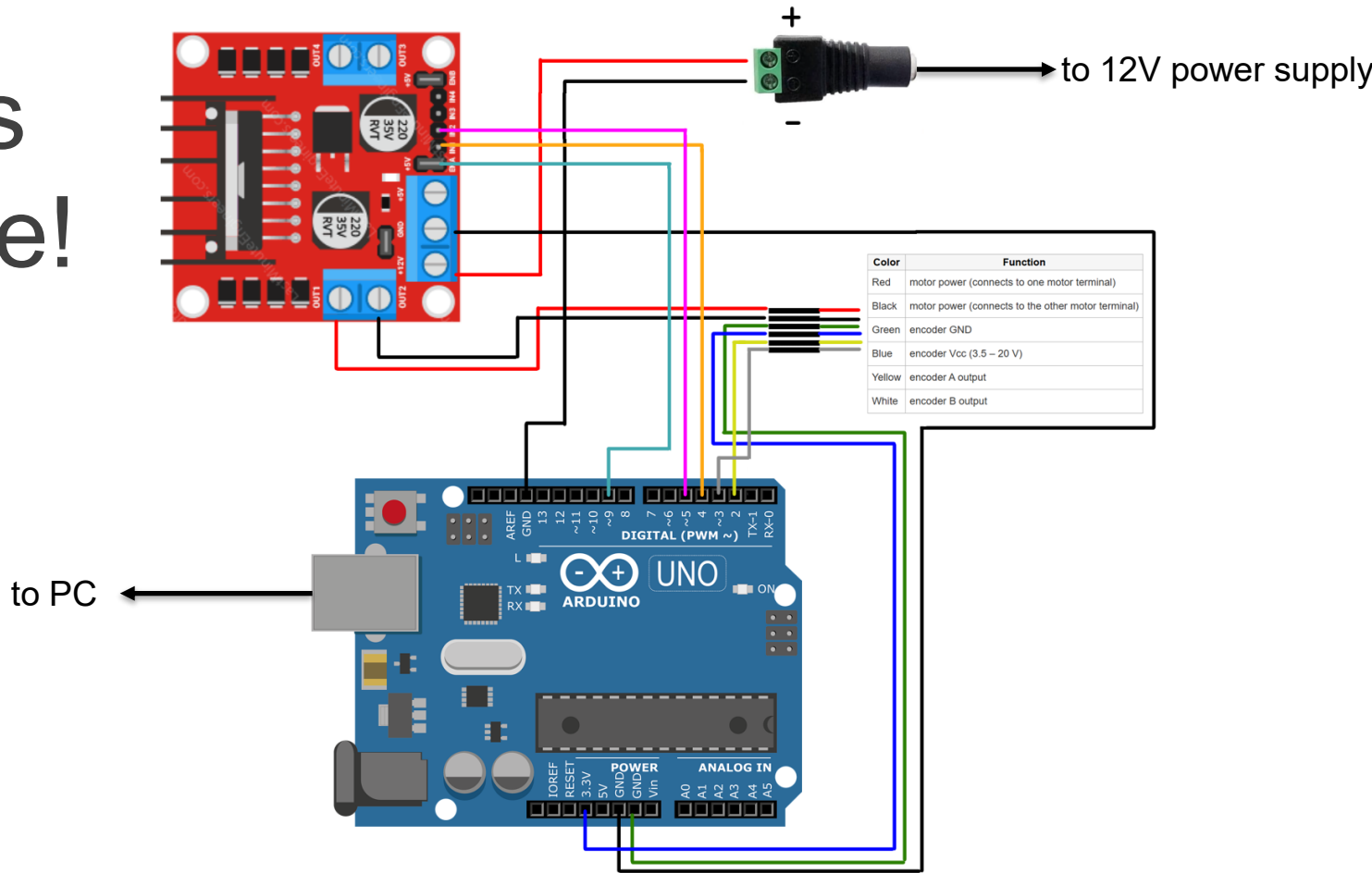
- This script will print the encoder position (in counts) to the Serial monitor
- **Ln 9:** Using the “Encoder” class, we create an *instance* called “myEncoder” that passes “encA” and “encB” as *arguments* to the class’s constructor
- **This is an object declaration and initiation**
- Now you may use the object to return encoder counts object by saying myEncoder.read();

```
1 //testing out the encoder
2
3 #include <Encoder.h> //this lets the compiler know we are using the "Encoder" library in this script
4 #define ENCODER_OPTIMIZE_INTERRUPTS //lets us use hardware interrupts to stay on top of where encoder is at
5
6 const int encA = 2;
7 const int encB = 3;
8
9 Encoder myEncoder(encA, encB);
10
11 long position = 0; //this will show us the encoder ticks
12
13
14 void setup() {
15     // put your setup code here, to run once:
16     Serial.begin(115200); //begin serial communication
17
18 }
19
20 void loop() {
21     // put your main code here, to run repeatedly:
22     position = myEncoder.read();
23
24
25     Serial.print("Counts: ");
26     Serial.println(position);
27
28 }
29
```


Using the Encoder

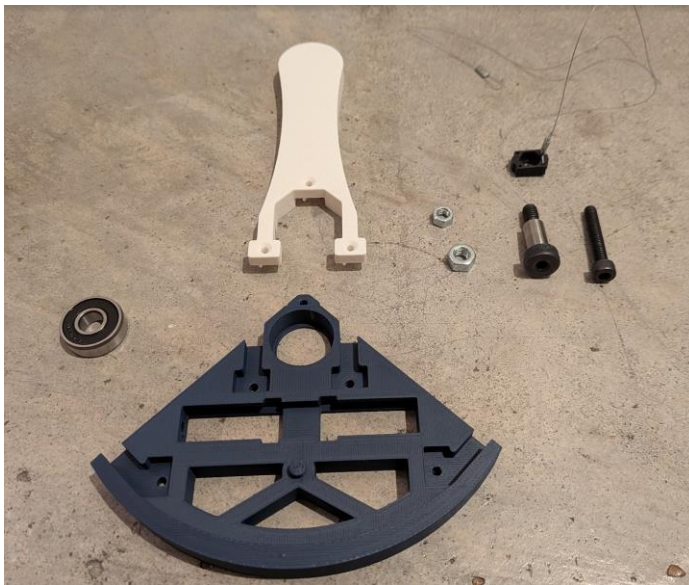
- Now try to get the encoder to readout in:
 - Degrees
 - Radians (2π radians = 360 degrees)

Wiring is complete!



Assemble the Handle and Sector

- Handle snaps into sector
 - Can secure with screws when they arrive...
- Ball bearing press-fits into sector



Transmissions

- Why use transmissions?
 - *Transmit* power from one location to another
 - *e.g.* driveshaft in a car, chain on a bike
 - Trade torque for velocity
 - The right *transmission ratio* will help translate the motor's output to the output needed in a device



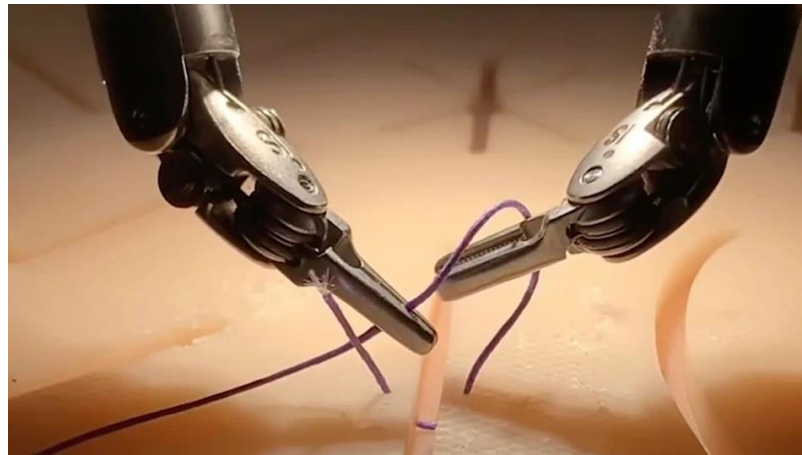
Torque and Gearing

- Which bike would you set a speed record on?
- Which could you actually ride up a hill with?
- We'll come back to this later



Cable Transmissions (Pulley-Like)

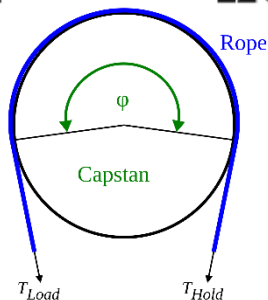
- Cable transmissions are often used in haptics and robotics
 - Very low friction
 - Very low *backlash*
 - Very stiff compared to belt-drive
 - These attributes make them highly “transparent”
 - Unfortunately they can be a bit of a pain, as you will find out shortly...



The Capstan

- This is what we will be using
- Has exponentially more holding force as it has more turns
- Works according to the equation:

$$T_{\text{load}} = T_{\text{hold}} e^{\mu\phi}$$

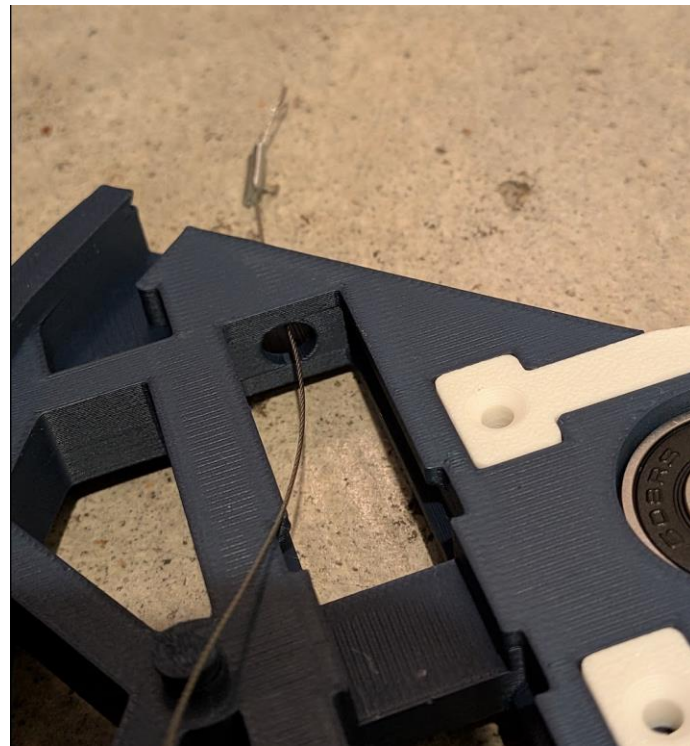
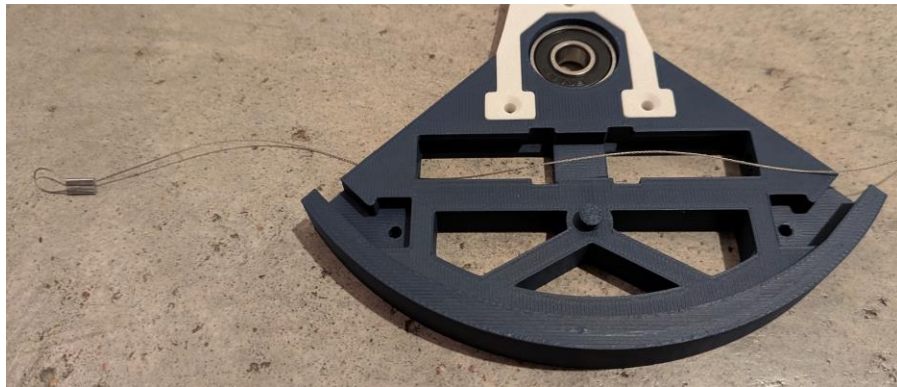


$$e^{\mu\phi} =$$

Number of turns	Coefficient of friction μ						
	0.1	0.2	0.3	0.4	0.5	0.6	0.7
0.5	1.4	1.9	2.6	3.5	4.8	6.6	9
1	1.9	3.5	6.6	12	23	43	81
2	3.5	12	43	152	535	1 881	6 661
3	6.6	43	286	1 881	12 392	81 612	537 503
4	12	152	1 881	23 228	286 751	3 540 026	43 702 631
5	23	535	12 392	286 751	6 635 624	153 552 935	3 553 321 281

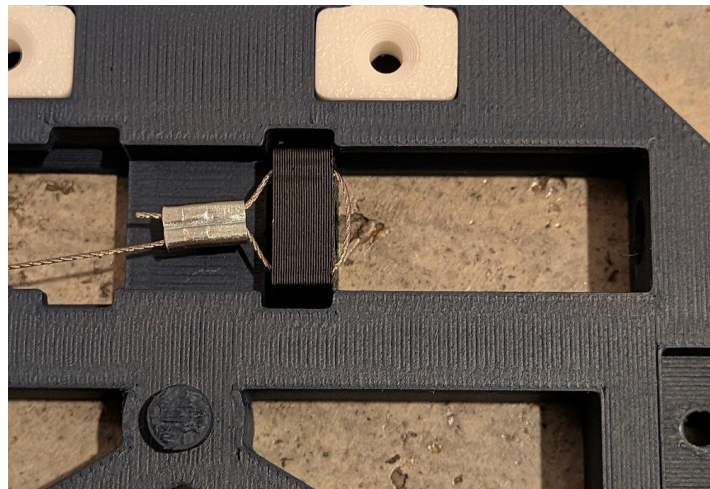
Prepare the tensioning mechanism

- Thread the loop end of the tension slide out of one side of the sector



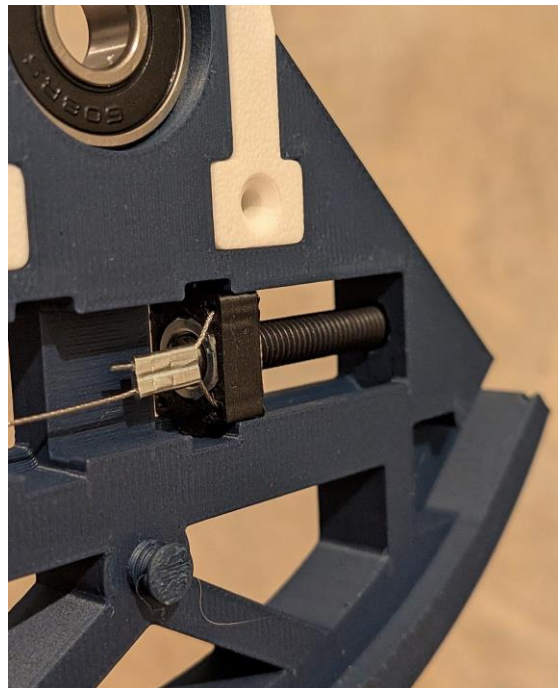
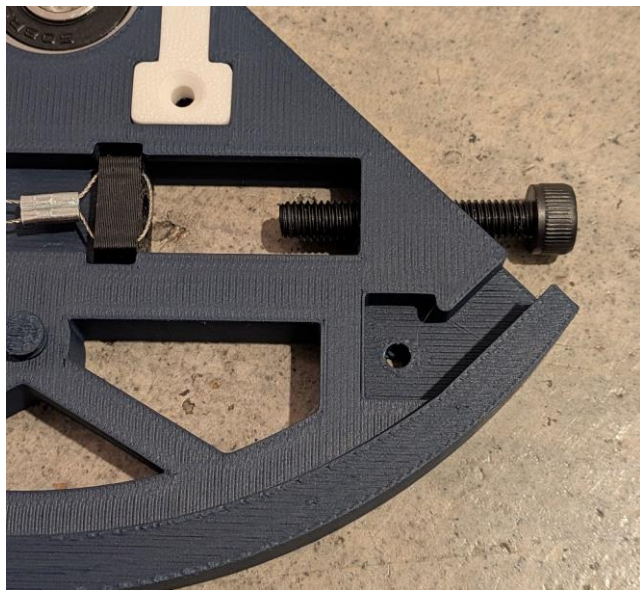
Prepare the tensioning mechanism

- Put the M5 nut in the tension slider
- Place the slider into the slot opposite the side of the sector you threaded the loop end through



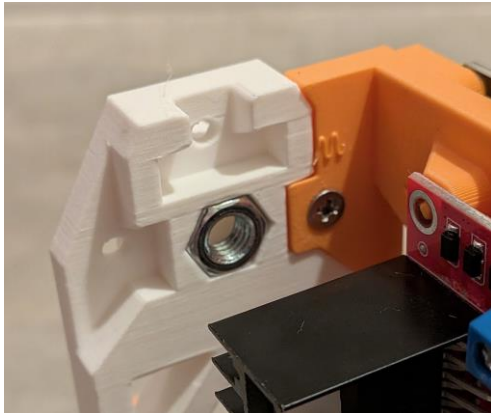
Prepare the tensioning mechanism

- Thread the M5 socket head bolt into the tension slider



Attach Sector and Handle to the Base

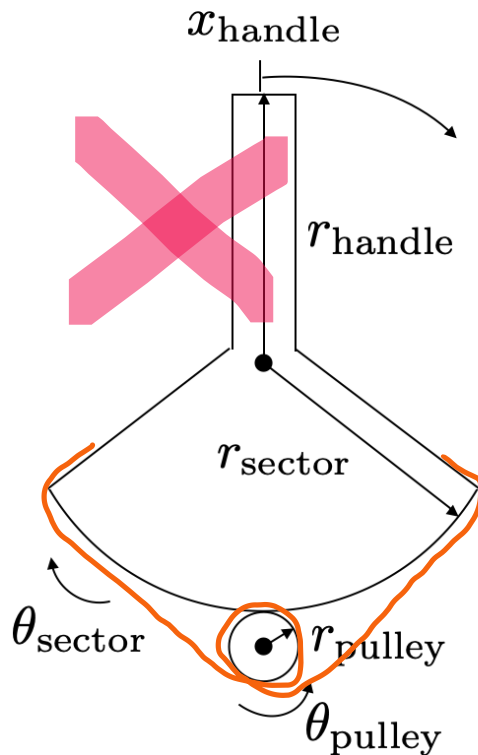
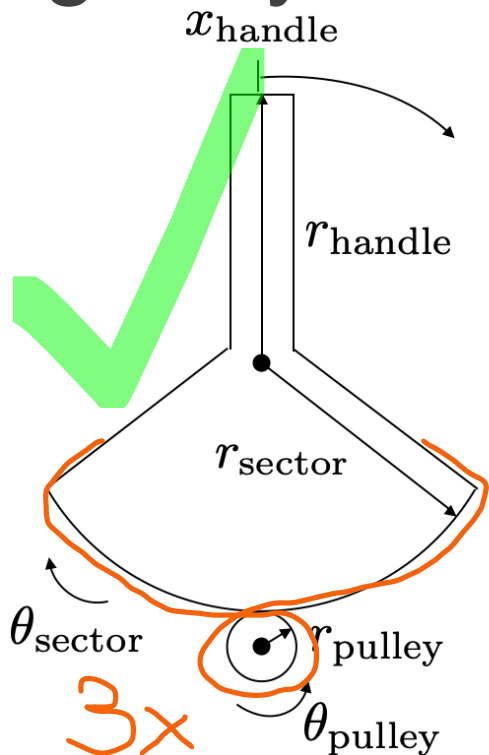
- Use the M6 shoulder screw and M6 nut to secure the handle/sector to the base
- You can tighten it down, but it is possible to over-tighten. Make sure the handle still spins freely on its bearing



Configure the Capstan Transmission

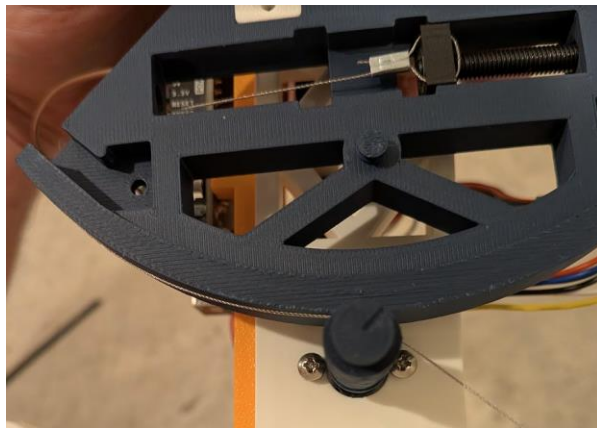
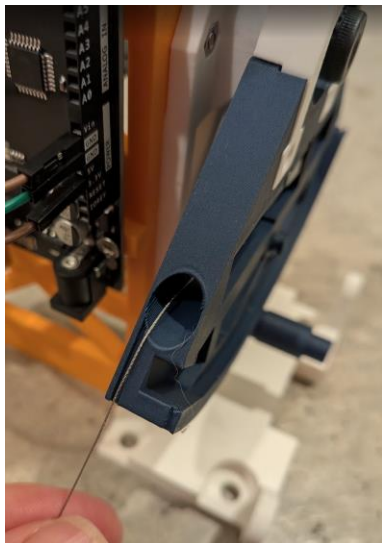
- This is the most difficult part of the whole process
- Be patient and accept that you may need to start over a few times
- It helps to work with a partner
 - Have partner hold the sector still while you route the cable

Capstan Cable Routing – Right and Wrong Way



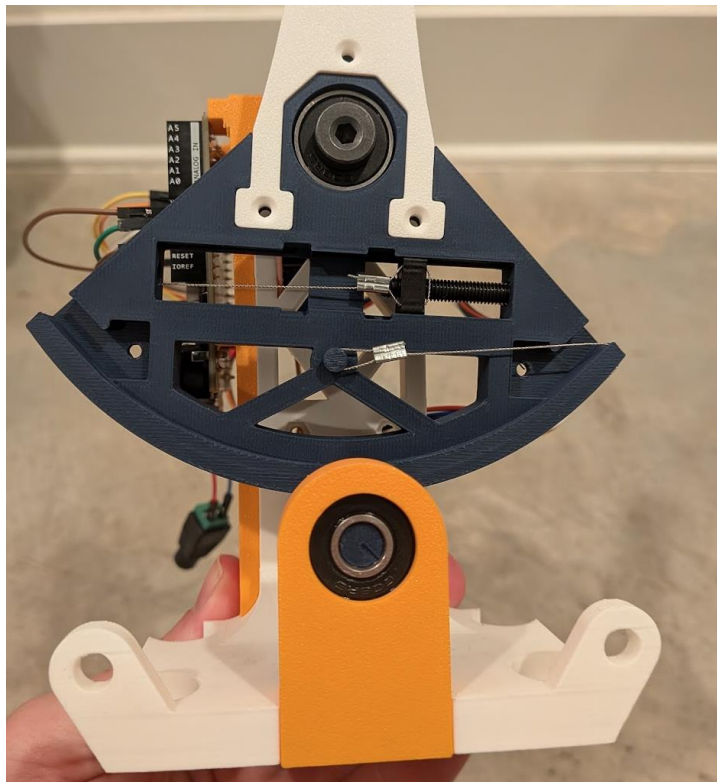
Capstan Transmission

- Wrap the cable **three times** around the motor shaft, through the notch on the other side, and secure it to the peg at the center of the sector
 - May need to loosen/tighten the M5 bolt to be able to reach the peg and hold some tension
 - If your cable is too long/short to reach we can get you another one



Support the motor shaft with the bearing housing

- Slide the feet of the bearing housing into the base to support the outer end of the motor shaft
- Now you can tighten/tweak the cable until the transmission is smooth and has good holding power
 - Once again, it is possible to over-tighten. Experimentation will help find a good balance.



If you make it to this point, try to:

- Write a sketch that converts encoder ticks to displacement at the end of the handle
 - Handle distance to axis of rotation: 85mm
 - Radius of motor shaft: 5mm
 - Radius of sector: 75mm
- Program the handle to behave like a spring
 - $F_{\text{spring}} = -kx$
- Return the **velocity** of the handle